15: C/Linux Interfacing I

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

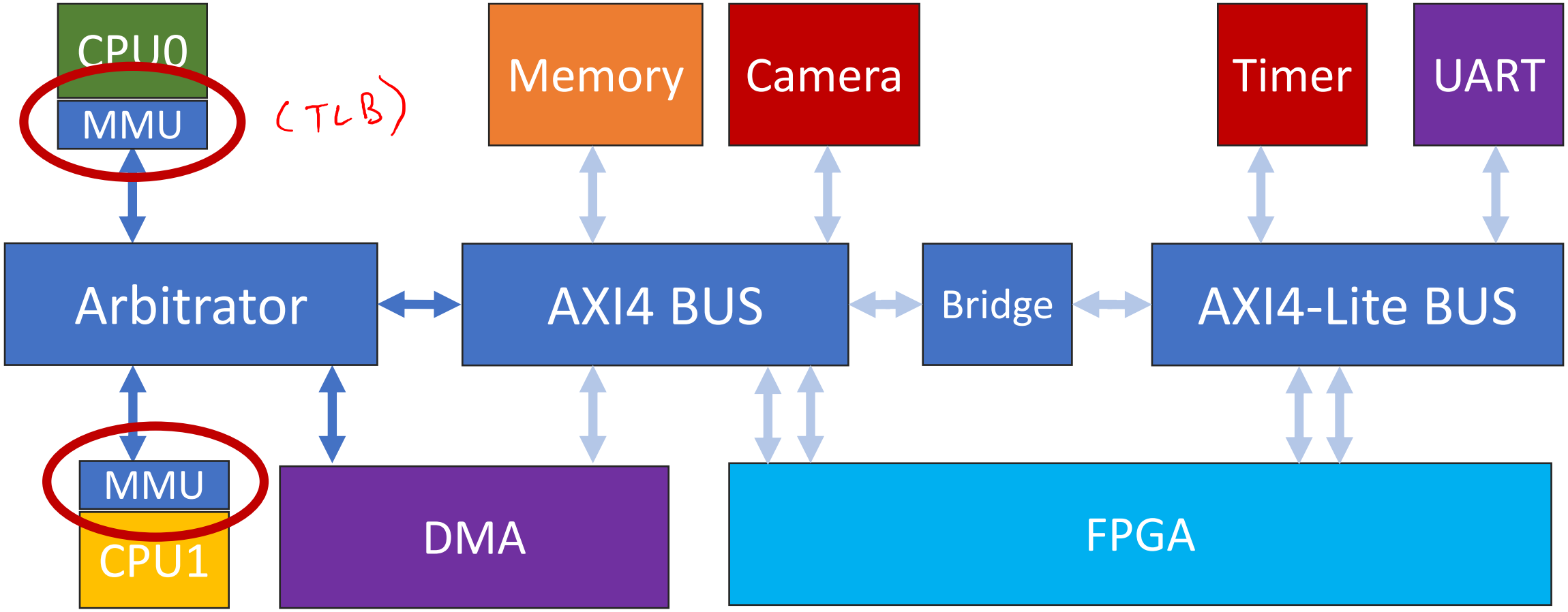
Some material taken from EECS370 at U. of Michigan

Python: ~60 seconds

Announcements

- P4 is due Friday (MMIO) ← 210 seconds
  - P4 doc fixed
- P5 is out (DMA) ← < 1 second
  - Adds popcount DMA interface
- Exam grading next week

Machine Model, V3



Virtual memory with an MMU

- MMU automatically translates each memory reference from a

virtual address

(which the programmer sees as a huge array of bytes)

to a

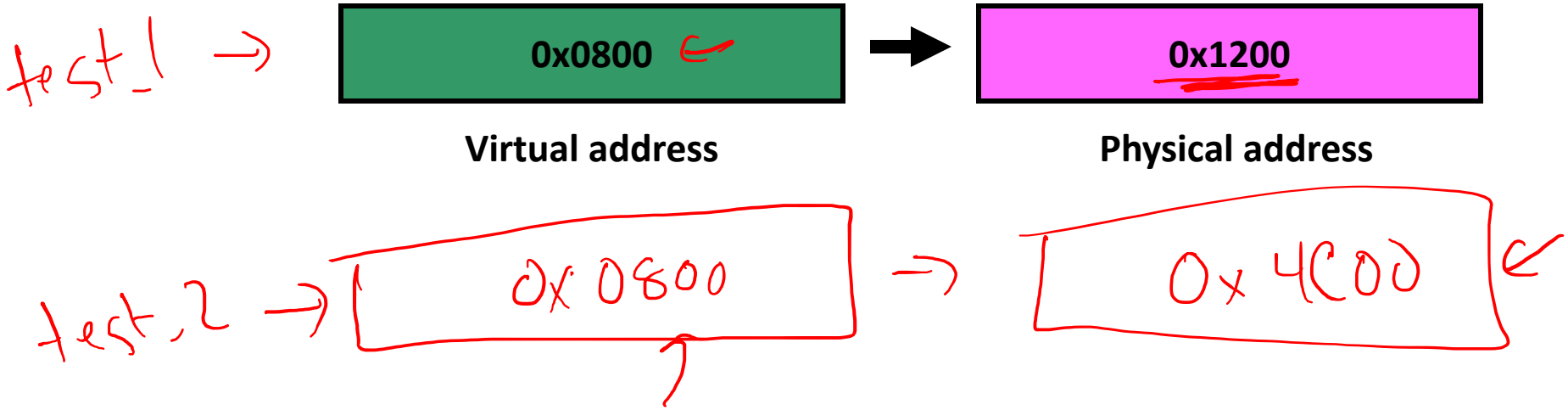
physical address

(which the hardware uses to identify where the storage actually resides)

Basics of Virtual Memory

- Any time you see the word virtual in computer science and architecture it means "using a level of indirection"

- Virtual memory hardware changes the virtual address the programmer sees into the physical one the memory chips see



MMU Address Translation

6000
7000

unity mapping

Virtual Address
(32-bit)

Virtual Base	Offset

0x69ff | 0004
2000 | 0008

v	Virtual Base	Physical Base
	69ff	4000
	3f00	4f00
	2000	2000
	6000	8000
	7000	8000

0x4000 0004
2000 0008

memory (mmio)

Physical Base	Offset

Physical Address

So how do we access the FPGA for P4?

- FPGA uses physical address
- CPU (w/Linux) uses virtual address

- Q: How do I talk to a physical address with Linux?

- A: Linux provides a special /dev/mem file to help us!

/dev/mem

- */dev/mem* is a character device file that is an image of the main memory of the computer. It may be used, for example, to examine (and even patch) the system.
- Byte addresses in */dev/mem* are interpreted as physical memory addresses. References to nonexistent locations cause errors to be returned.
- Requires root (sudo) access

Using /dev/mem in C

```
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int dev_mem_fd = -1;

int main (){
   dev_mem_fd = open("/dev/mem", O_RDWR|O_SYNC);

   if (dev_mem_fd < 0) {
      perror("open() /dev/mem"); //no root access
      return 0;
   }

   if (close(dev_mem_fd) != 0) {  perror("close()");   }
   dev_mem_fd = -1;

   return 0;
}
```

mmap – memory map

- **mmap**() creates a new mapping in the virtual address space of the calling process. The starting address for the new mapping is specified in *addr*.

- The contents of a file mapping are initialized using *length* bytes starting at offset *offset* in the file (or other object) referred to by the file descriptor *fd*.

Using mmap in C

```
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <unistd.h>

int main (){

   int dev_mem_fd = -1;
   void * vaddr_base; // <-- void pointer!

   dev_mem_fd = open("/dev/mem", O_RDWR|O_SYNC); //requires root!
   if (dev_mem_fd < 0) { perror("open() /dev/mem"); return 1; }

   // Map 1KB of physical memory starting at 0x40000000
   // to 1KB of virtual memory starting at vaddr_base
   vaddr_base = mmap(0, 1024, PROT_READ|PROT_WRITE,
         MAP_SHARED, dev_mem_fd, 0x40000000);

   if (vaddr_base == MAP_FAILED) { perror("mmap()"); return 1; }

   if (munmap(vaddr_base, 1024) != 0) { perror("munmap()"); }
   if (close(dev_mem_fd) != 0) { perror("close()"); }
   dev_mem_fd = -1;
   return 0;
}
```

vaddr_base = 0x3cf2

Simple Popcount w/C

```
#include <fcntl.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdio.h>
#include <sys/mman.h>
#include <unistd.h>

int main (){

   int dev_mem_fd = -1;
   void * vaddr_base;

   dev_mem_fd = open("/dev/mem", O_RDWR|O_SYNC);
   if (dev_mem_fd < 0) { perror("open() /dev/mem"); return 1; }

   // Map 1KB of physical memory starting at 0x40000000
   // to 1KB of virtual memory starting at vaddr_base
   vaddr_base = mmap(0, 1024, PROT_READ|PROT_WRITE,
         MAP_SHARED, dev_mem_fd, 0x40000000);
```

```
   if (vaddr_base == MAP_FAILED) { perror("mmap()"); return 1; }

   uint32_t * reset = (uint32_t*) vaddr_base;
   uint32_t * count = (uint32_t*) (vaddr_base + 0x4);

   *reset = 1; //reset count
   *count = 0xf; //add new value
   printf("Count: %d\n", *count); //read count

   if (munmap(vaddr_base, 1024) != 0) { perror("munmap()"); }
   if (close(dev_mem_fd) != 0) { perror("close()"); }
   dev_mem_fd = -1;

   return 0;
}
```

0x4000_0000
0x4000_0004

Userspace I/O

- Linux provides a more restrictive version of /dev/mem called /dev/uio
- Linux exposes a small portion of I/O memory space to the user through /dev/uioX interfaces.
- Treat it just like /dev/mem, but not the entire memory space

The Pynq provides /dev/uio0

```
$ cat /sys/class/uio/uio0/maps/map0/addr
0x40000000

$cat /sys/class/uio/uio0/maps/map0/size
0x00010000    (8KB)
```

Userspace Popcount w/C

```
#include <fcntl.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdio.h>
#include <sys/mman.h>
#include <unistd.h>

int main (){

   int dev_mem_fd = -1;
   void * vaddr_base;

   //Mapping user-space I/O
   dev_mem_fd = open("/dev/uio0", O_RDWR|O_SYNC);
   if (dev_mem_fd < 0) { perror("open() /dev/uio0"); return 1; }

   // Map 1KB of physical memory starting at 0x40000000
   // to 1KB of virtual memory starting at vaddr_base
   vaddr_base = mmap(0, 1024, PROT_READ|PROT_WRITE,
         MAP_SHARED, dev_mem_fd, 0x0); // not 0x40000000
   if (vaddr_base == MAP_FAILED) { perror("mmap()"); return 1; }

   uint32_t * reset = (uint32_t*) vaddr_base;
   uint32_t * count = (uint32_t*) (vaddr_base + 0x4);

   *reset = 1; //reset count
   *count = 0xf; //add new value
   printf("Count: %d\n", *count); //read count

   if (munmap(vaddr_base, 1024) != 0) { perror("munmap()"); }
   if (close(dev_mem_fd) != 0) { perror("close()"); }
   dev_mem_fd = -1;

   return 0;
}
```

MMIO Popcount

- Python

```
Timing 'medium.bin'
Found 4196661 Ones
Total Time:7.867364578007255 seconds
```

- C

$ time sudo ./popcount_mmio_devmem medium.bin

Counted 4196661 ones

real 0m0.143s

Next Time:

- DMA interfacing with C

15: C/Linux Interfacing I

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University